

CHAPTER 10

PRINCIPLES OF DIMENSIONAL MODELING

CHAPTER OBJECTIVES

- Clearly understand how the requirements definition determines data design
- Introduce dimensional modeling and contrast it with entity-relationship modeling
- Review the basics of the STAR schema
- Find out what is inside the fact table and inside the dimension tables
- Determine the advantages of the STAR schema for data warehouses
- Review examples of the STAR schema

FROM REQUIREMENTS TO DATA DESIGN

The requirements definition completely drives the data design for the data warehouse. Data design consists of putting together the data structures. A group of data elements form a data structure. Logical data design includes determination of the various data elements that are needed and combination of the data elements into structures of data. Logical data design also includes establishing the relationships among the data structures.

Let us look at Figure 10-1. Notice how the phases start with requirements gathering. The results of the requirements gathering phase is documented in detail in the requirements definition document. An essential component of this document is the set of information package diagrams. Remember that these are information matrices showing the metrics, business dimensions, and the hierarchies within individual business dimensions.

The information package diagrams form the basis for the logical data design for a data warehouse made up of data marts. The data design process results in a dimensional data model.

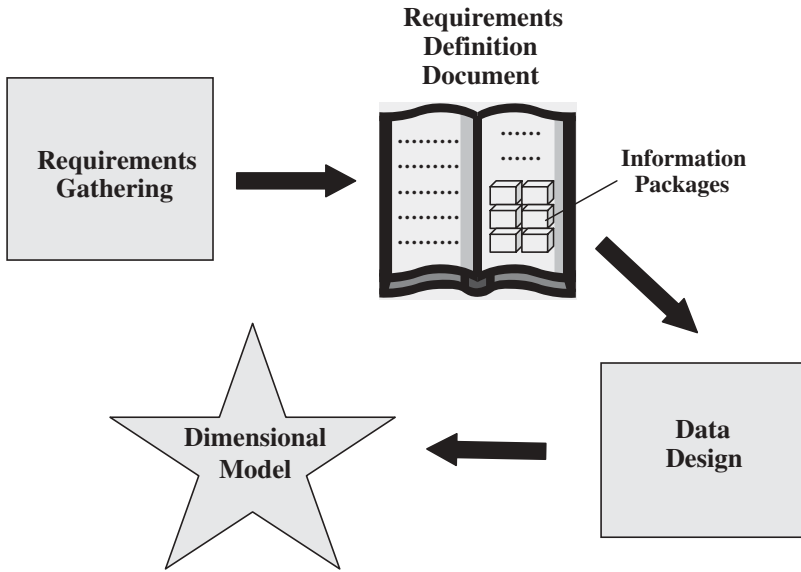


Figure 10-1 From requirements to data design.

Design Decisions

Before we proceed with designing the dimensional data model, let us quickly review some of the design decisions you have to make:

Choosing the Process. Selecting the subjects from the information packages for the first set of logical structures to be designed.

Choosing the Grain. Determining the level of detail for the data in the data structures.

Identifying and Conforming the Dimensions. Choosing the business dimensions (such as product, market, time, etc.) to be included in the first set of structures and making sure that each particular data element in every business dimension is conformed to one another.

Choosing the Facts. Selecting the metrics or units of measurements (such as product sale units, dollar sales, dollar revenue, etc.) to be included in the first set of structures.

Choosing the Duration of the Database. Determining how far back in time you should go for historical data.

Dimensional Modeling Basics

Dimensional modeling gets its name from the business dimensions we need to incorporate into the logical data model. It is a logical design technique to structure the business dimensions and the metrics that are analyzed along these dimensions. This modeling technique is intuitive for that purpose. The model has also proved to provide high performance for queries and analysis.

The multidimensional information package diagram we have discussed is the foundation for the dimensional model. Therefore, the dimensional model consists of the specific data

structures needed to represent the business dimensions. These data structures also contain the metrics or facts.

In Chapter 5, we discussed information package diagrams in sufficient detail. We specifically looked at an information package diagram for automaker sales. Please go back and review Figure 5-5 in that chapter. What do you see? In the bottom section of the diagram, you observe the list of measurements or metrics that the automaker wants to use for analysis. Next, look at the column headings. These are the business dimensions along which the automaker wants to analyze the measurements or metrics. Under each column heading you see the dimension hierarchies and categories within that business dimension. What you see under each column heading are the attributes relating to that business dimension.

Reviewing the information package diagram for automaker sales, we notice three types of data entities: (1) measurements or metrics, (2) business dimensions, and (3) attributes for each business dimension. So, when we put together the dimensional model to represent the information contained in the automaker sales information package, we need to come up with data structures to represent these three types of data entities. Let us discuss how we can do this.

First, let us work with the measurements or metrics seen at the bottom of the information package diagram. These are the facts for analysis. In the automaker sales diagram, the facts are as follows:

Actual sale price
MSRP
Options price
Full price
Dealer add-ons
Dealer credits
Dealer invoice
Amount of down payment
Manufacturer proceeds
Amount financed

Each of these data items is a measurement or fact. Actual sale price is a fact about what the actual price was for the sale. Full price is a fact about what the full price was relating to the sale. As we review each of these factual items, we find that we can group all of these into a single data structure. In relational database terminology, you may call the data structure a relational table. So, the metrics or facts from the information package diagram will form the fact table. For the automaker sales analysis this fact table would be the automaker sales fact table.

Look at Figure 10-2 showing how the fact table is formed. The fact table gets its name from the subject for analysis; in this case, it is automaker sales. Each fact item or measurement goes into the fact table as an attribute for automaker sales.

We have determined one of the data structures to be included in the dimensional model for automaker sales and derived the fact table from the information package diagram. Let us now move on to the other sections of the information package diagram, taking the business dimensions, one by one. Look at the product business dimension in Figure 5-5.

The product business dimension is used when we want to analyze the facts by products. Sometimes our analysis could be a breakdown by individual models. Another analysis could

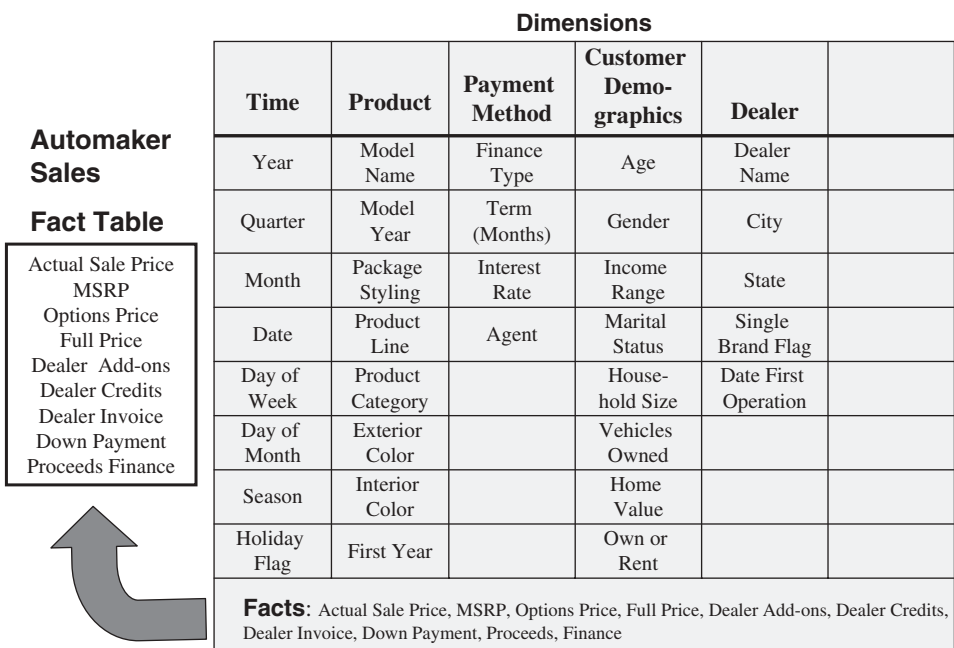


Figure 10-2 Formation of the automaker sales fact table.

be at a higher level by product lines. Yet another analysis could be at even a higher level by product categories. The list of data items relating to the product dimension are as follows:

- Model name
- Model year
- Package styling
- Product line
- Product category
- Exterior color
- Interior color
- First model year

What can we do with all these data items in our dimensional model? All of these relate to the product in some way. We can, therefore, group all of these data items in one data structure or one relational table. We can call this table the product dimension table. The data items in the above list would all be attributes in this table.

Looking further into the information package diagram, we note the other business dimensions shown as column headings. In the case of the automaker sales information package diagram, these other business dimensions are dealer, customer demographics, payment method, and time. Just as we formed the product dimension table, we can form the remaining dimension tables of dealer, customer demographics, payment method, and time. The data items shown within each column would then be the attributes for each corresponding dimension table.

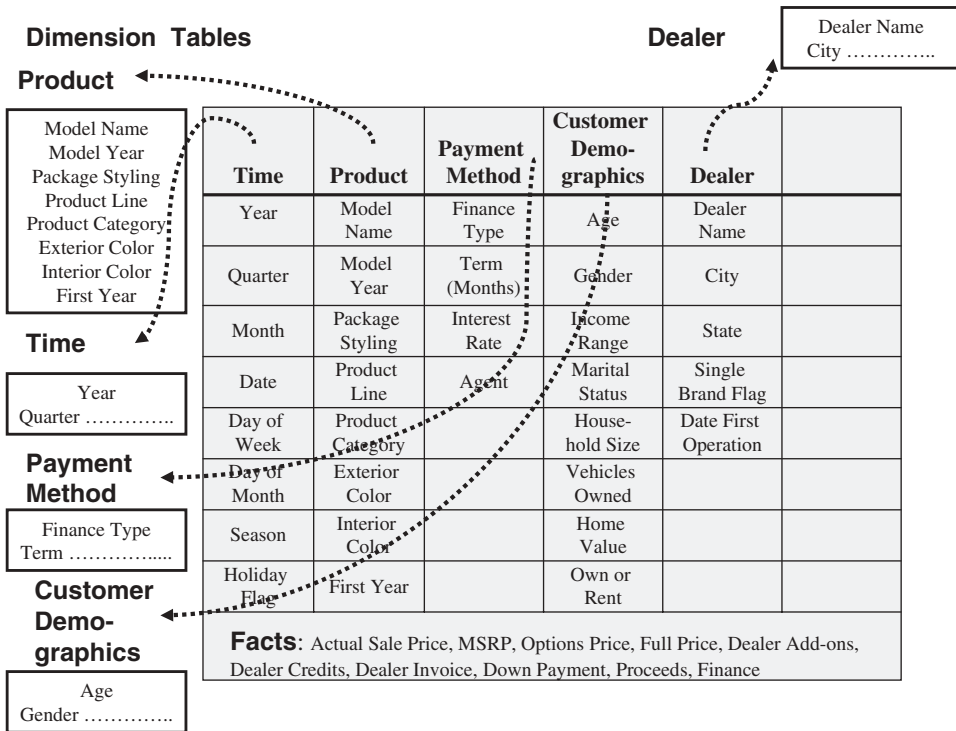


Figure 10-3 Formation of the automaker dimension tables.

Figure 10-3 puts all of this together. It shows how the various dimension tables are formed from the information package diagram. Look at the figure closely and see how each dimension table is formed.

So far we have formed the fact table and the dimension tables. How should these tables be arranged in the dimensional model? What are the relationships and how should we mark the relationships in the model? The dimensional model should primarily facilitate queries and analyses. What would be the types of queries and analyses? These would be queries and analyses where the metrics inside the fact table are analyzed across one or more dimensions using the dimension table attributes.

Let us examine a typical query against the automaker sales data. How much sales proceeds did the Jeep Cherokee, Year 2007 Model with standard options, generate in July 2007 at Big Sam Auto dealership for buyers who own their homes and who took 3-year leases, financed by Daimler-Chrysler Financing? We are analyzing actual sale price, MSRP, and full price. We are analyzing these facts along attributes in the various dimension tables. The attributes in the dimension tables act as constraints and filters in our queries. We also find that any or all of the attributes of each dimension table can participate in a query. Further, each dimension table has an equal chance to be part of a query.

Before we decide how to arrange the fact and dimension tables in our dimensional model and mark the relationships, let us go over what the dimensional model needs to achieve and what its purposes are. Here are some of the criteria for combining the tables into a dimensional model.

- The model should provide the best data access.
- The whole model must be query-centric.
- It must be optimized for queries and analyses.
- The model must show that the dimension tables interact with the fact table.
- It should also be structured in such a way that every dimension can interact equally with the fact table.
- The model should allow drilling down or rolling up along dimension hierarchies.

With these requirements, we find that a dimensional model with the fact table in the middle and the dimension tables arranged around the fact table satisfies the conditions. In this arrangement, each of the dimension tables has a direct relationship with the fact table in the middle. This is necessary because every dimension table with its attributes must have an even chance of participating in a query to analyze the attributes in the fact table.

Such an arrangement in the dimensional model looks like a star formation, with the fact table at the core of the star and the dimension tables along the spikes of the star. The dimensional model is therefore called a STAR schema.

Let us examine the STAR schema for the automaker sales as shown in Figure 10-4. The sales fact table is in the center. Around this fact table are the dimension tables of product, dealer, customer demographics, payment method, and time. Each dimension table is related to the fact table in a one-to-many relationship. In other words, for one row in the product dimension table, there are one or more related rows in the fact table.

E-R Modeling Versus Dimensional Modeling

We are familiar with data modeling for operational or OLTP systems. We adopt the Entity-Relationship (E-R) modeling technique to create the data models for these systems.

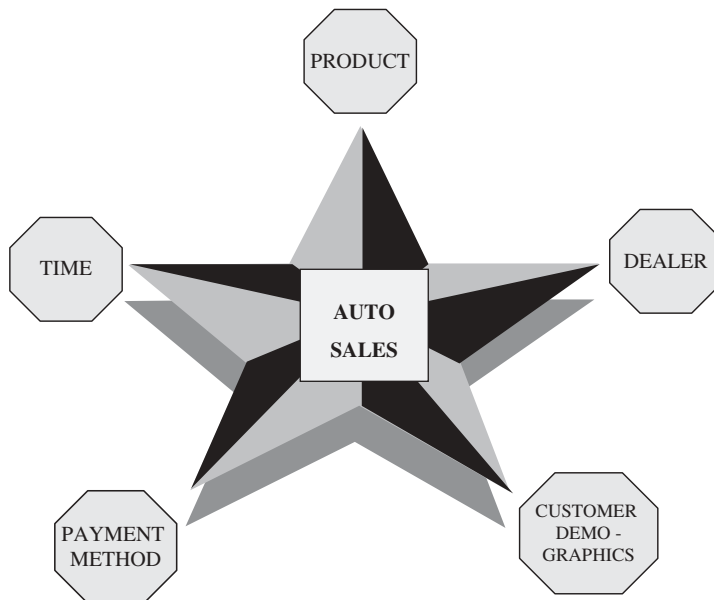


Figure 10-4 STAR schema for automaker sales.

- ◆ OLTP systems capture details of events or transactions
- ◆ OLTP systems focus on individual events
- ◆ An OLTP system is a window into micro-level transactions
- ◆ Picture at detail level necessary to run the business
- ◆ Suitable only for questions at transaction level
- ◆ Data consistency, non-redundancy, and efficient data storage critical

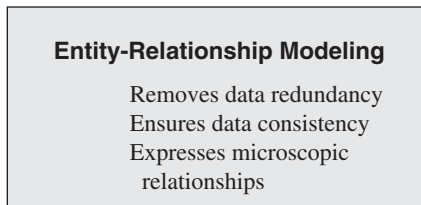


Figure 10-5 E-R modeling for OLTP systems

Figure 10-5 lists the characteristics of OLTP systems and shows why E-R modeling is suitable for OLTP systems.

We have so far discussed the basics of the dimensional model and find that this model is most suitable for modeling the data for the data warehouse. Let us recapitulate the characteristics of the data warehouse information and review how dimensional modeling is suitable for this purpose. Study the features of dimensional modeling listed in Figure 10-6.

- ◆ DW meant to answer questions on overall process
- ◆ DW focus is on how managers view the business
- ◆ DW reveals business trends
- ◆ Information is centered around a business process
- ◆ Answers show how the business measures the process
- ◆ The measures to be studied in many ways along several business dimensions

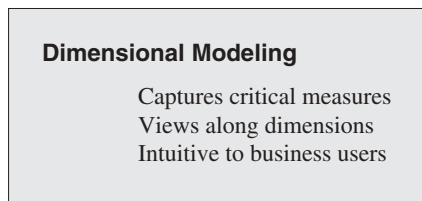


Figure 10-6 Dimensional modeling for the data warehouse.

Use of CASE Tools

Many case tools are available for data modeling. In Chapter 8, we introduced these tools and their features. You can use these tools for creating the logical schema and the physical schema for specific target database management systems (DBMSs).

You can use a case tool to define the tables, the attributes, and the relationships. You can assign the primary keys and indicate the foreign keys. You can form the entity-relationship diagrams. All of this is done very easily using graphical user interfaces and powerful drag-and-drop facilities. After creating an initial model, you may add fields, delete fields, change field characteristics, create new relationships, and make any number of revisions with utmost ease.

Another very useful function found in the case tools is the ability to forward-engineer the model and generate the schema for the target database system you need to work with. Forward-engineering is easily done with these case tools.

For modeling the data warehouse, we are interested in the dimensional modeling technique. Most of the existing vendors have expanded their modeling case tools to include dimensional modeling. You can create fact tables, dimension tables, and establish the relationships between each dimension table and the fact table. The result is a STAR schema for your model. Again, you can forward-engineer the dimensional STAR model into a relational schema for your chosen database management system.

THE STAR SCHEMA

Now that you have been introduced to the STAR schema, let us take a simple example and examine its characteristics in detail. Creating the STAR schema is the fundamental data design technique for the data warehouse. It is necessary to gain a good grasp of this technique.

Review of a Simple STAR Schema

We will take a simple STAR schema designed for order analysis. Assume this to be the schema for a manufacturing company and that the marketing department is interested in determining how they are doing with the orders received by the company.

Figure 10-7 shows this simple STAR schema. It consists of the orders fact table shown in the middle of the schema diagram. Surrounding the fact table are the four dimension tables of customer, salesperson, order date, and product. Let us begin to examine this STAR schema. Look at the structure from the point of view of the marketing department. The users in this department will analyze the orders using dollar amounts, cost, profit margin, and sold quantity. This information is found in the fact table of the structure. The users will analyze these measurements by breaking down the numbers in combinations by customer, salesperson, date, and product. All these dimensions along which the users will analyze are found in the structure. The STAR schema structure is a structure that can be easily understood by the users and with which they can comfortably work. The structure mirrors how the users normally view their critical measures along their business dimensions.

When you look at the order dollars, the STAR schema structure intuitively answers the questions of what, when, by whom, and to whom. From the STAR schema, the users can easily visualize the answers to these questions: For a given amount of dollars, what was

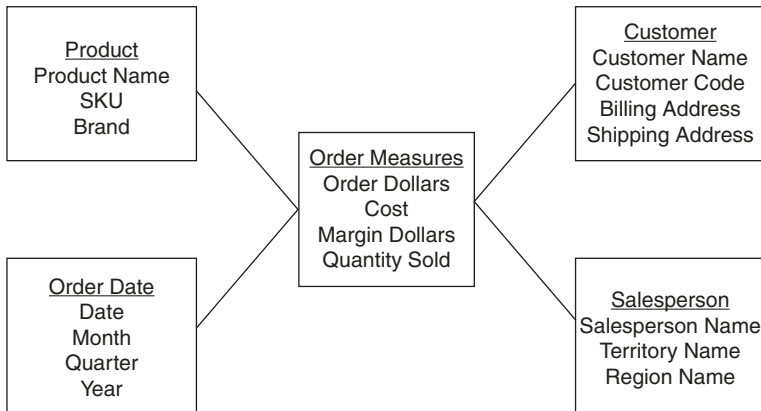


Figure 10-7 Simple STAR schema for orders analysis.

the product sold? Who was the customer? Which salesperson brought the order? When was the order placed?

When a query is made against the data warehouse, the results of the query are produced by combining or joining one or more dimension tables with the fact table. The joins are between the fact table and individual dimension tables. The relationship of a particular row in the fact table is with the rows in each dimension table. These individual relationships are clearly shown as the spikes of the STAR schema.

Take a simple query against the STAR schema. Let us say that the marketing department wants the quantity sold and order dollars for product *bigpart-1*, relating to customers in the

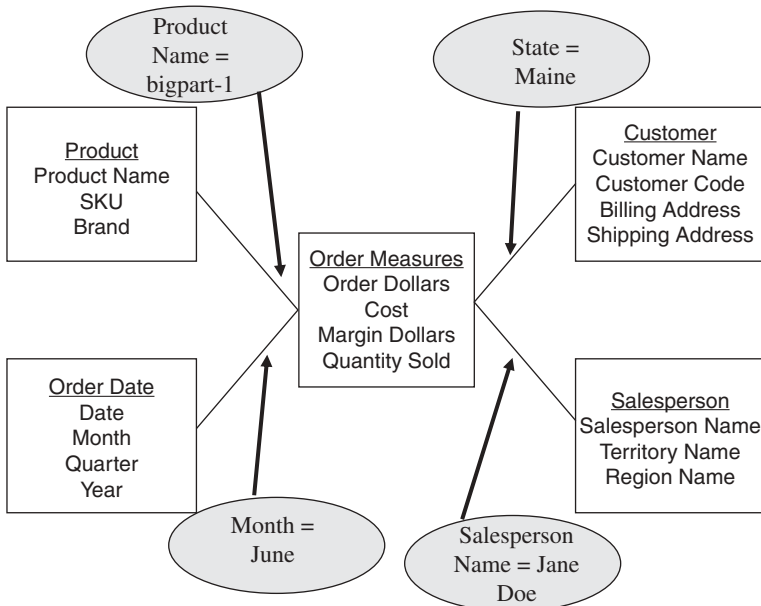


Figure 10-8 Understanding a query from the STAR schema.

state of *Maine*, obtained by salesperson *Jane Doe*, during the month of *June*. Figure 10-8 shows how this query is formulated from the STAR schema. Constraints and filters for queries are easily understood by looking at the STAR schema.

A common type of analysis is the drilling down of summary numbers to get at the details at the lower levels. Let us say that the marketing department has initiated a specific analysis by placing the following query: Show me the total quantity sold of product brand *big parts* to customers in the *northeast region for year 2008*. In the next step of the analysis, the marketing department now wants to drill down to the level of *quarters in 2008* for the *northeast region* for the same product brand, *big parts*. Next, the analysis goes down to the level of individual products in that brand. Finally, the analysis goes to the level of details by individual states in the northeast region. The users can easily discern all of this drill-down analysis by reviewing the STAR schema. Refer to Figure 10-9 to see how the drill-down is derived from the STAR schema.

Inside a Dimension Table

We have seen that a key component of the STAR schema is the set of dimension tables. These dimension tables represent the business dimensions along which the metrics are

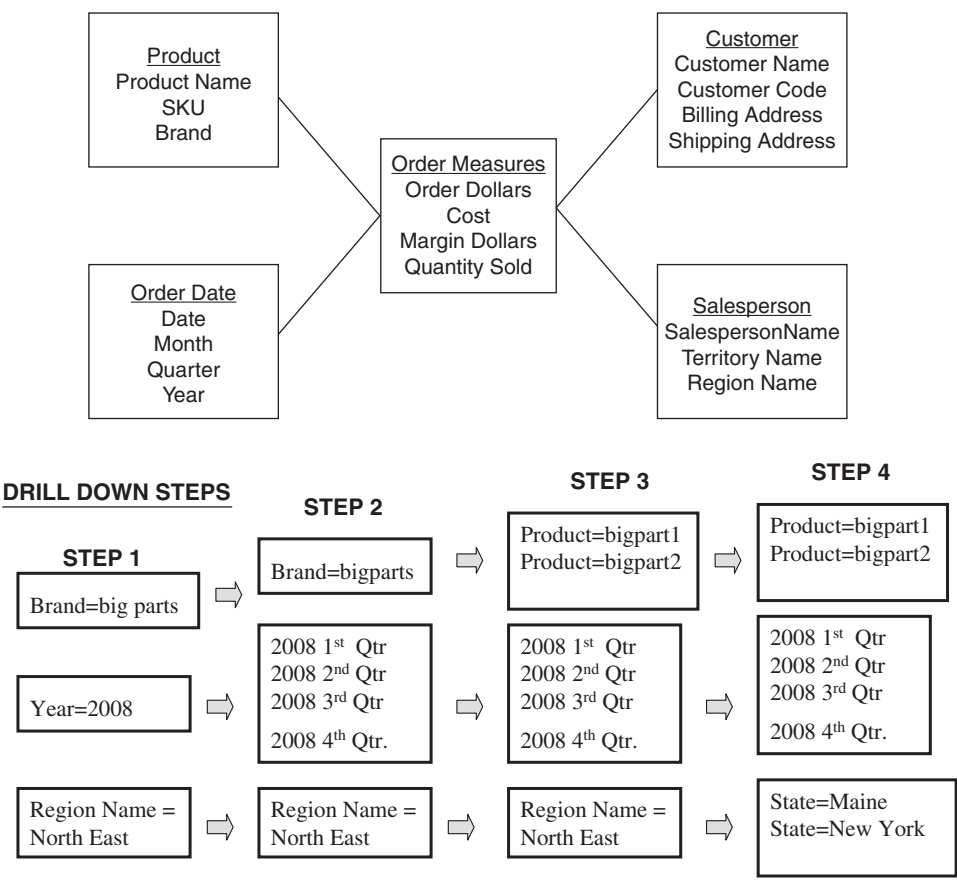


Figure 10-9 Understanding drill-down analysis from the STAR schema.

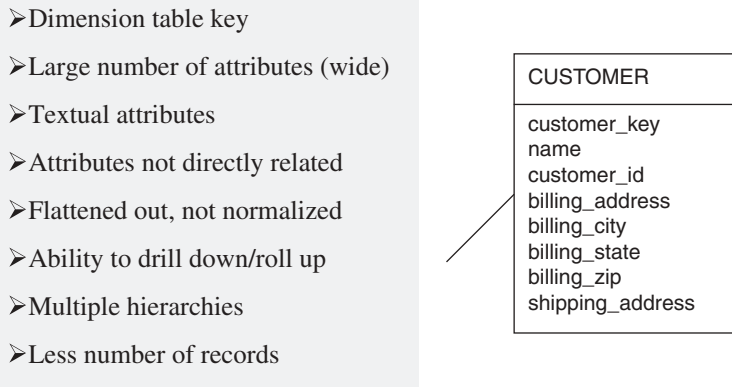


Figure 10-10 Inside a dimension table.

analyzed. Let us look inside a dimension table and study its characteristics. See Figure 10-10 and review the following observations.

Dimension Table Key. The primary key of the dimension table uniquely identifies each row in the table.

Table is Wide. Typically, a dimension table has many columns or attributes. It is not uncommon for some dimension tables to have more than 50 attributes. Therefore, we say that the dimension table is wide. If you lay it out as a table with columns and rows, the table is spread out horizontally.

Textual Attributes. In the dimension table you will seldom find any numerical values used for calculations. The attributes in a dimension table are of textual format.

These attributes represent the textual descriptions of the components within the business dimensions. Users will compose their queries using these descriptors.

Attributes not Directly Related. Frequently you will find that some of the attributes in a dimension table are not directly related to the other attributes in the table. For example, package size is not directly related to product brand; nevertheless, package size and product brand could both be attributes of the product dimension table.

Not Normalized. The attributes in a dimension table are used over and over again in queries. An attribute is taken as a constraint in a query and applied directly to the metrics in the fact table. For efficient query performance, it is best if the query picks up an attribute from the dimension table and goes directly to the fact table and not through other intermediary tables. If you normalize the dimension table, you will be creating such intermediary tables and that will not be efficient. Therefore, a dimension table is flattened out, not normalized.

Drilling Down, Rolling Up. The attributes in a dimension table provide the ability to get to the details from higher levels of aggregation to lower levels of details. For example, the three attributes zip code, city, and state form a hierarchy. You may get the total sales by state, then drill down to total sales by city, and then by zip code. Going the other way, you may first get the totals by zip codes, and then roll up to totals by city and state.

Multiple Hierarchies. In the example of the customer dimension, there is a single hierarchy going up from individual customer to zip, city, and state. But dimension tables often provide for multiple hierarchies, so that drilling down may be performed along

any of the multiple hierarchies. Take for example a product dimension table for a department store. In this business, the marketing department may have its way of classifying the products into product categories and product departments. On the other hand, the accounting department may group the products differently into categories and product departments. So in this case, the product dimension table will have the attributes of marketing–product–category, marketing–product–department, finance–product–category, and finance–product–department.

Fewer Records. A dimension table typically has fewer records or rows than the fact table. A product dimension table for an automaker may have just 500 rows. On the other hand, the fact table may contain millions of rows.

Inside the Fact Table

Let us now get into a fact table and examine the components. Remember this is where we keep the measurements. We may keep the details at the lowest possible level. In the department store fact table for sales analysis, we may keep the units sold by individual transactions at the cashier’s checkout. Some fact tables may just contain summary data. These are called aggregate fact tables. Figure 10-11 lists the characteristics of a fact table. Let us review these characteristics.

Concatenated Key. A row in the fact table relates to a combination of rows from all the dimension tables. In this example of a fact table, you find quantity ordered as an attribute. Let us say the dimension tables are product, time, customer, and sales representative. For these dimension tables, assume that the lowest level in the dimension hierarchies are individual product, a calendar date, a specific customer, and a single sales representative. Then a single row in the fact table must relate to a particular product, a specific calendar date, a specific customer, and an individual sales representative. This means the row in the fact table must be identified by the primary keys of these four dimension tables. Thus, the primary key of the fact table must be the concatenation of the primary keys of all the dimension tables.

Data Grain. This is an important characteristic of the fact table. As we know, the data grain is the level of detail for the measurements or metrics. In this example, the metrics are at the detailed level. The quantity ordered relates to the quantity of a particular product on a single order, on a certain date, for a specific customer, and procured by a

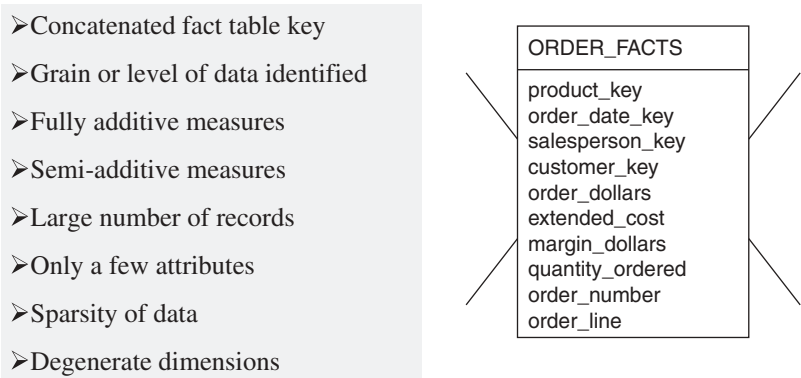


Figure 10-11 Inside a fact table.

specific sales representative. If we keep the quantity ordered as the quantity of a specific product for each month, then the data grain is different and is at a higher level.

Fully Additive Measures. Let us look at the attributes *order_dollars*, *extended_cost*, and *quantity_ordered*. Each of these relates to a particular product on a certain date for a specific customer procured by an individual sales representative. In a certain query, let us say that the user wants the totals for the particular product on a certain date, not for a specific customer, but for customers in a particular state. Then we need to find all the rows in the fact table relating to all the customers in that state and add the *order_dollars*, *extended_cost*, and *quantity_ordered* to come up with the totals. The values of these attributes may be summed up by simple addition. Such measures are known as fully additive measures. Aggregation of fully additive measures is done by simple addition. When we run queries to aggregate measures in the fact table, we will have to make sure that these measures are fully additive. Otherwise, the aggregated numbers may not show the correct totals.

Semiadditive Measures. Consider the *margin_dollars* attribute in the fact table. For example, if the *order_dollars* is 120 and *extended_cost* is 100, the *margin_percentage* is 20. This is a calculated metric derived from the *order_dollars* and *extended_cost*. If you are aggregating the numbers from rows in the fact table relating to all the customers in a particular state, you cannot add up the *margin_percentages* from all these rows and come up with the aggregated number. Derived attributes such as *margin_percentage* are not additive. They are known as semiadditive measures. Distinguish semiadditive measures from fully additive measures when you perform aggregations in queries.

Table Deep, Not Wide. Typically a fact table contains fewer attributes than a dimension table. Usually, there are about 10 attributes or less. But the number of records in a fact table is very large in comparison. Take a very simplistic example of 3 products, 5 customers, 30 days, and 10 sales representatives represented as rows in the dimension tables. Even in this example, the number of fact table rows will be 4500, very large in comparison with the dimension table rows. If you lay the fact table out as a two-dimensional table, you will note that the fact table is narrow, with a small number of columns, but very deep, with a large number of rows.

Sparse Data. We have said that a single row in the fact table relates to a particular product, a specific calendar date, a specific customer, and an individual sales representative. In other words, for a particular product, a specific calendar date, a specific customer, and an individual sales representative, there is a corresponding row in the fact table. What happens when the date represents a closed holiday and no orders are received and processed? The fact table rows for such dates will not have values for the measures. Also, there could be other combinations of dimension table attributes, values for which the fact table rows will have null measures. Do we need to keep such rows with null measures in the fact table? There is no need for this. Therefore, it is important to realize this type of sparse data and understand that the fact table could have gaps.

Degenerate Dimensions. Look closely at the example of the fact table. You find the attributes of *order_number* and *order_line*. These are not measures or metrics or facts. Then why are these attributes in the fact table? When you pick up attributes for the dimension tables and the fact tables from operational systems, you will be left with some data elements in the operational systems that are neither facts nor strictly dimension attributes. Examples of such attributes are reference numbers like order numbers, invoice numbers, order line numbers, and so on. These attributes are useful in some

types of analyses. For example, you may be looking for average number of products per order. Then you will have to relate the products to the order number to calculate the average. Attributes such as *order_number* and *order_line* in the example are called degenerate dimensions and these are kept as attributes of the fact table.

The Factless Fact Table

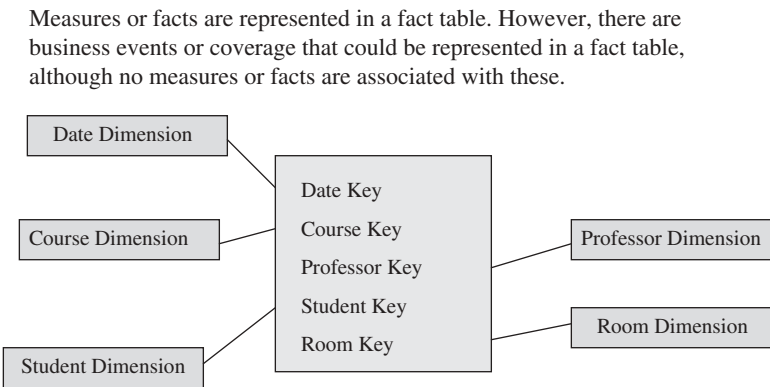
Apart from the concatenated primary key, a fact table contains facts or measures. Let us say we are building a fact table to track the attendance of students. For analyzing student attendance, the possible dimensions are student, course, date, room, and professor. The attendance may be affected by any of these dimensions. When you want to mark the attendance relating to a particular course, date, room, and professor, what is the measurement you come up with for recording the event? In the fact table row, the attendance will be indicated with the number *one*. Every fact table row will contain the number *one* as attendance. If so, why bother to record the number *one* in every fact table row? There is no need to do this. The very presence of a corresponding fact table row could indicate the attendance. This type of situation arises when the fact table represents events. Such fact tables really do not need to contain facts. They are *factless* fact tables. Figure 10-12 shows a typical factless fact table.

Data Granularity

By now, we know that granularity represents the level of detail in the fact table. If the fact table is at the lowest grain, then the facts or metrics are at the lowest possible level at which they could be captured from the operational systems. What are the advantages of keeping the fact table at the lowest grain? What is the trade-off?

When you keep the fact table at the lowest grain, users can drill down to the lowest level of detail from the data warehouse without the need to go to the operational systems themselves. Base level fact tables must be at the natural lowest levels of all corresponding dimensions. By doing this, queries for drill down and roll up can be performed efficiently.

What then are the natural lowest levels of the corresponding dimensions? In the example with the dimensions of product, date, customer, and sales representative, the natural lowest levels are an individual product, a specific individual date, an individual customer, and an



Tracks the attendance although no measured facts in the fact table

Figure 10-12 A factless fact table.

individual sales representative, respectively. So, in this case, a single row in the fact table should contain measurements at the lowest level for an individual product, ordered on a specific date, relating to an individual customer, and procured by an individual sales representative.

Let us say we want to add a new attribute of district in the sales representative dimension. This change will not warrant any changes in the fact table rows because these are already at the lowest level of individual sales representative. This is a “graceful” change because all the old queries will continue to run without any changes. Similarly, let us assume we want to add a new dimension of promotion. Now you will have to recast the fact table rows to include promotion dimensions. Still, the fact table grain will be at the lowest level.

Even here, the old queries will still run without any changes. This is also a “graceful” change. Fact tables at the lowest grain facilitate “graceful” extensions.

But we have to pay the price in terms of storage and maintenance for the fact table at the lowest grain. Lowest grain necessarily means large numbers of fact table rows. In practice, however, we build aggregate fact tables to support queries looking for summary numbers.

There are two more advantages of granular fact tables. Granular fact tables serve as natural destinations for current operational data that may be extracted frequently from operational systems. Further, the more recent data mining applications need details at the lowest grain. Data warehouses feed data into data mining applications.

STAR SCHEMA KEYS

Figure 10-13 illustrates how the keys are formed for the dimension and fact tables.

Primary Keys

Each row in a dimension table is identified by a unique value of an attribute designated as the primary key of the dimension. In a product dimension table, the primary key identifies each product uniquely. In the customer dimension table, the customer number identifies each

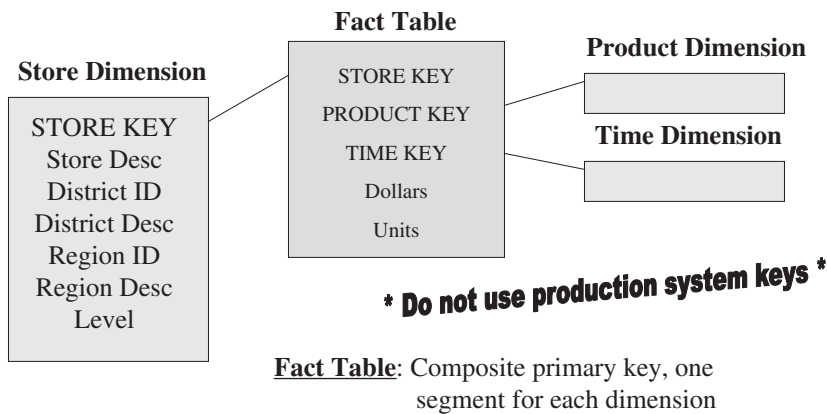


Figure 10-13 The STAR schema keys.

customer uniquely. Similarly, in the sales representative dimension table, the social security number of the sales representative identifies each sales representative.

We have picked these out as possible candidate keys for the dimension tables. Now let us consider some implications of these candidate keys. Let us assume that the product code in the operational system is an 8-position code, two of which positions indicate the code of the warehouse where the product is normally stored, and two other positions denote the product category. Let us see what happens if we use the operational system product code as the primary key for the product dimension table.

The data warehouse contains historic data. Assume that the product code gets changed in the middle of a year, because the product is now stored in a different warehouse of the company. So we have to change the product code in the data warehouse. If the product code is the primary key of the product dimension table, then the newer data for the same product will reside in the data warehouse with different key values. This could cause problems if we need to aggregate the data from before the change with the data from after the change to the product code. What really has caused this problem? The problem is the result of our decision to use the operational system key as the key for the dimension table.

Surrogate Keys

How do we resolve the problem faced in the previous section? Can we use production system keys as primary keys for dimension tables? If not, what are the other candidate keys?

There are two general principles to be applied when choosing primary keys for dimension tables. The first principle is derived from the problem caused when the product began to be stored in a different warehouse. In other words, the product key in the operational system has built-in meanings. Some positions in the operational system product key indicate the warehouse and some other positions in the key indicate the product category. These are built-in meanings in the key. The first principle to follow is: avoid built-in meanings in the primary key of the dimension tables.

In some companies, a few of the customers are no longer listed with the companies. They could have left their respective companies many years ago. It is possible that the customer numbers of such discontinued customers are reassigned to new customers. Now, let us say we had used the operational system customer key as the primary key for the customer dimension table. We will have a problem because the same customer number could relate to the data for the newer customer and also to the data of the retired customer. The data of the retired customer may still be used for aggregations and comparisons by city and state. Therefore, the second principle is: do not use production system keys as primary keys for dimension tables.

What then should we use as primary keys for dimension tables? The answer is to use surrogate keys. The surrogate keys are simply system-generated sequence numbers. They do not have any built-in meanings. Of course, the surrogate keys will be mapped to the production system keys. Nevertheless, they are different. The general practice is to keep the operational system keys as additional attributes in the dimension tables. Refer back to Figure 10-13. The STORE KEY is the surrogate primary key for the store dimension table. The operational system primary key for the store reference table may be kept as just another non-key attribute in the store dimension table.

Foreign Keys

Each dimension table is in a one-to-many relationship with the central fact table. So the primary key of each dimension table must be a foreign key in the fact table. If there are

four dimension tables of product, date, customer, and sales representative, then the primary key of each of these four tables must be present in the orders fact table as foreign keys.

Let us reexamine the primary keys for the fact tables. There are three options:

1. *A single compound primary key whose length is the total length of the keys of the individual dimension tables.* Under this option, in addition to the compound primary key, the foreign keys must also be kept in the fact table as additional attributes. This option increases the size of the fact table.
2. *A concatenated primary key that is the concatenation of all the primary keys of the dimension tables.* Here you need not keep the primary keys of the dimension tables as additional attributes to serve as foreign keys. The individual parts of the primary keys themselves will serve as the foreign keys.
3. *A generated primary key independent of the keys of the dimension tables.* In addition to the generated primary key, the foreign keys must also be kept in the fact table as additional attributes. This option also increases the size of the fact table.

In practice, option (2) is used in most fact tables. This option enables you to easily relate the fact table rows with the dimension table rows.

ADVANTAGES OF THE STAR SCHEMA

When you look at the STAR schema, you find that it is simply a relational model with a one-to-many relationship between each dimension table and the fact table. What is so special about the arrangement of the STAR schema? Why is it declared to be eminently suitable for the data warehouse? What are the reasons for its wide use and success in providing optimization for processing queries?

Although the STAR schema is a relational model, it is not a normalized model. The dimension tables are purposely denormalized. This is a basic difference between the STAR schema and relational schemas for OLTP systems.

Before we discuss some very significant advantages of the STAR schema, we need to be aware that strict adherence to this arrangement is not always the best option. For example, if customer is one of the dimensions and if the enterprise has a very large number of customers, a denormalized customer dimension table is not desirable. A large dimension table may increase the size of the fact table correspondingly.

However, the advantages far outweigh any shortcomings. So, let us go over the advantages of the STAR schema.

Easy for Users to Understand

Users of OLTP systems interact with the applications through predefined GUI screens or preset query templates. There is practically no need for the users to understand the data structures behind the scenes. The data structures and the database schema remain in the realm of the IT professionals.

Users of decision support systems such as data warehouses are different. Here the users themselves will formulate queries. When they interact with the data warehouse through third-party query tools, the users should know what to ask for. They must gain a familiarity with what data is available to them in the data warehouse. They must have an understanding of the data structures and how the various pieces are associated with one another in the overall scheme. They must comprehend the connections without difficulty.

The STAR schema reflects exactly how the users think and need data for querying and analysis. They think in terms of significant business metrics. The fact table contains the metrics. The users think in terms of business dimensions for analyzing the metrics. The dimension tables contain the attributes along which the users normally query and analyze. When you explain to the users that the units of product A are stored in the fact table and point out the relationship of this piece of data to each dimension table, the users readily understand the connections. That is because the STAR schema defines the join paths in exactly the same way users normally visualize the relationships. The STAR schema is intuitively understood by the users.

Try to walk a user through the relational schema of an OLTP system. For them to understand the connections, you will have to take them through a maze of normalized tables, sometimes passing through several tables, one by one, to get even the smallest result set. The STAR schema emerges as a clear winner because of its simplicity. Users understand the structures and the connections very easily.

The STAR schema has definite advantages after implementation. However, the advantages even in the development stage cannot be overlooked. Because the users understand the STAR schema so very well, it is easy to use it as a vehicle for communicating with the users during the development of the data warehouse.

Optimizes Navigation

In a database schema, what is the purpose of the relationships or connections among the data entities? The relationships are used to go from one table to another for obtaining the information you are looking for. The relationships provide the ability to navigate through the database. You hop from table to table using the join paths.

If the join paths are numerous and convoluted, your navigation through the database gets difficult and slow. On the other hand, if the join paths are simple and straightforward, your navigation is optimized and becomes faster.

A major advantage of the STAR schema is that it optimizes the navigation through the database. Even when you are looking for a query result that is seemingly complex, the navigation is still simple and straightforward. Let us look at an example and understand how this works. Look at Figure 10-14 showing a STAR schema for analyzing defects in automobiles. Assume you are the service manager at an automobile dealership selling GM automobiles. You noticed a high incidence of chipped white paint on the Corvettes in January 2009. You need a tool to analyze such defects, determine the underlying causes, and resolve the problems.

In the STAR schema, the number of defects is kept as metrics in the middle as part of the defects fact table. The time dimension contains the model year. The component dimension has part information; for example, pearl white paint. The problem dimension carries the types of problems; for example, chipped paint. The product dimension contains the make, model, and trim package of the automobiles. The supplier dimension contains data on the suppliers of parts.

Now see how easy it is to determine the supplier causing the chipped paint on the pearl white Corvettes. Look at the four arrows pointing to the fact table from the four dimension tables. These arrows show how you will navigate to the rows in the fact table by isolating the Corvette from the product dimension, chipped paint from the problem dimension, pearl white paint from the component dimension, and January 2009 from the time dimension. From the fact table, the navigation goes directly to the supplier dimension to isolate the supplier causing the problem.

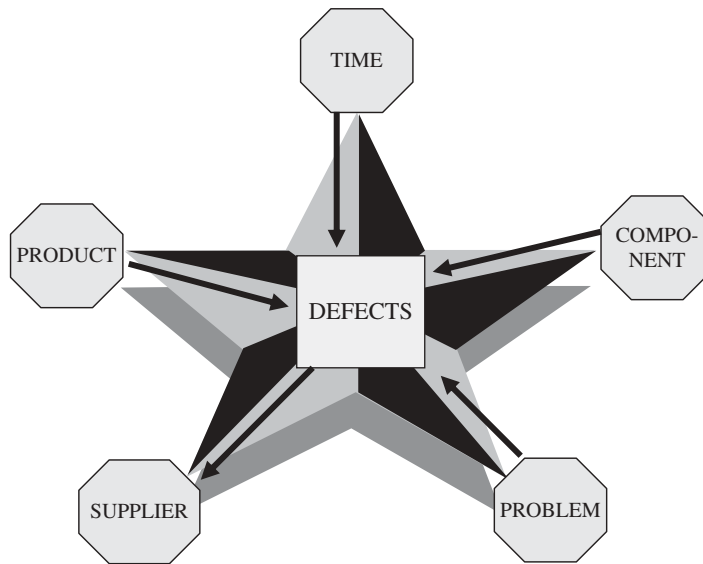


Figure 10-14 The STAR schema optimizes navigation.

Most Suitable for Query Processing

We have already mentioned a few times that the STAR schema is a query-centric structure. This means that the STAR schema is most suitable for query processing. Let us see how this is true.

Let us form a simple query on the STAR schema for the order analysis shown in Figure 10-7. What is the total extended cost of product A sold to customers in San Francisco during January 2009? This is a three-dimensional query. What should be the characteristics of the data structure or schema if it is to be most suitable for processing this query? The final result, which is the total extended cost, will come from the rows in the fact table. But from which rows? The answer is those rows relating to product A, relating to customers in San Francisco, and relating to January 2009.

Let us see how the query will be processed. First, select the rows from the customer dimension table where the city is San Francisco. Then, from the fact table, select only those rows that are related to these customer dimension rows. This is the first result set of rows from the fact tables. Next, select the rows in the Time dimension table where the month is January 2009. Select from the first result set only those rows that are related to these time dimension rows. This is now the second result set of fact table rows. Move on to the next dimension of product. Select the rows in the product dimension table where the product is product A. Select from the second result only those rows that are related to the selected product dimension rows. You now have the final result of fact table rows. Add up the extended cost to get the total.

Irrespective of the number of dimensions that participate in the query and irrespective of the complexity of the query, every query is simply executed first by selecting rows from the dimension tables using the filters based on the query parameters and then finding the corresponding fact table rows. This is possible because of the simple and straightforward join paths and because of the very arrangement of the STAR schema. There is no intermediary maze to be navigated to reach the fact table from the dimension tables.

Another important aspect of data warehouse queries is the ability to drill down or roll up. Let us quickly run through a drill-down scenario. Let us say we have queried and obtained the total extended cost for all the customers in the state of California. The result comes from the set of fact table rows. Then we want to drill down and look at the results by zip code ranges. This is obtained by making a further selection from the selected fact table rows relating to the chosen zip code ranges. Drill down is a process of further selection of the fact table rows. Going the other way, rolling up is a process of expanding the selection of the fact table rows.

STARjoin and STARindex

The STAR schema allows the query processor software to use better execution plans. It enables specific performance schemes to be applied to queries. The STAR schema arrangement is eminently suitable for special performance techniques such as the STARjoin and the STARindex.

STARjoin is a high-speed, single-pass, parallelizable, multitable join. It can join more than two tables in a single operation. This special scheme boosts query performance.

STARindex is a specialized index to accelerate join performance. These are indexes created on one or more foreign keys of the fact table. These indexes speed up joins between the dimension tables and the fact table.

Chapter 18 deals with the physical design and indexing methods for the data warehouse.

STAR SCHEMA: EXAMPLES

In this section we provide four examples of STAR schema relating to four different types of businesses. In each case, notice the following carefully and understand the STAR schema:

- The metrics or facts being analyzed
- The business dimensions used for analysis
- The hierarchy available within each dimension for drill down and roll up
- The primary and foreign keys

Video Rental

See Figure 10-15.

Supermarket

See Figure 10-16.

Wireless Phone Service

See Figure 10-17.

Auction Company

See Figure 10-18.

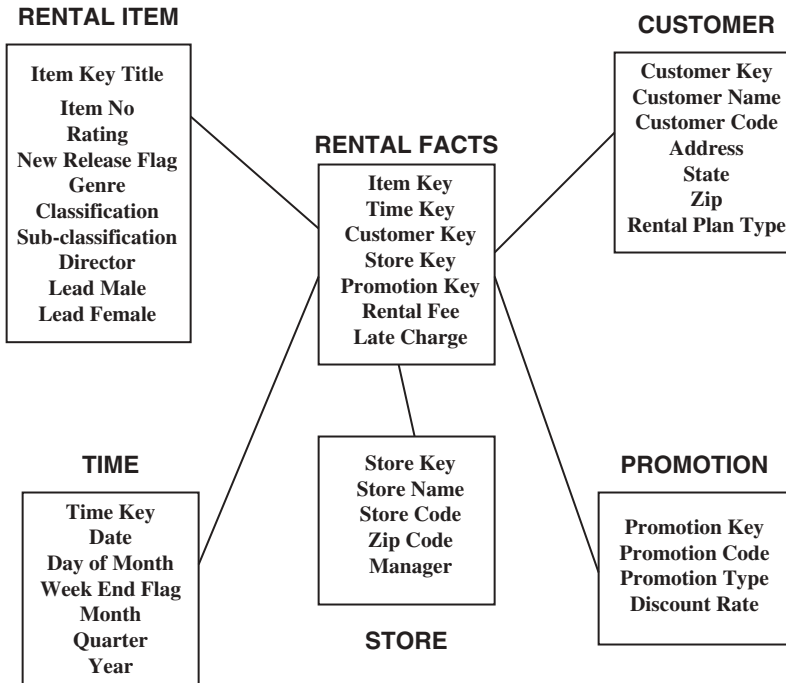


Figure 10-15 STAR schema example: video rental.

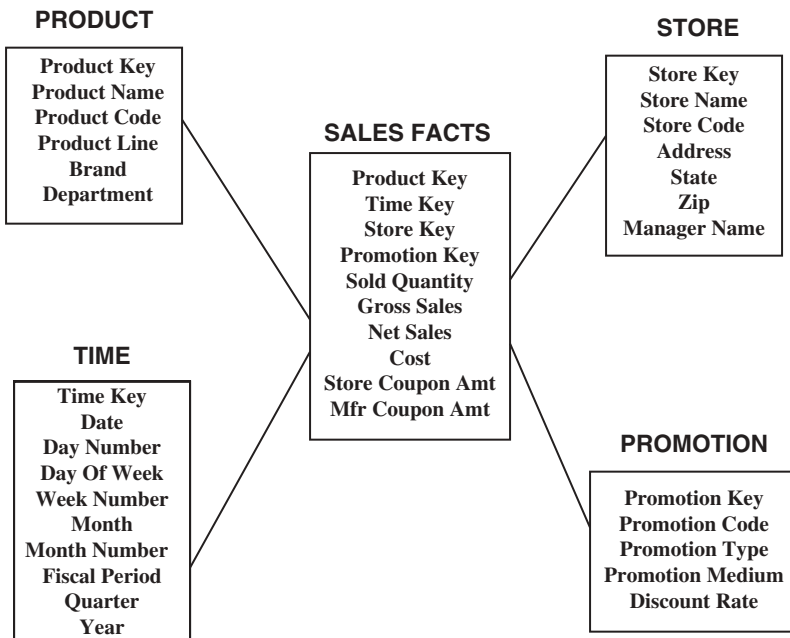


Figure 10-16 STAR schema example: supermarket.

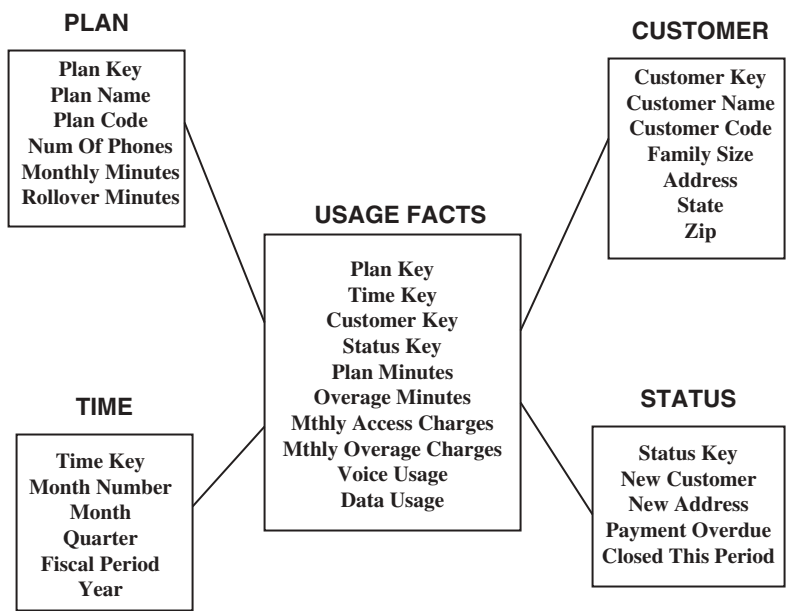


Figure 10-17 STAR schema example: wireless phone service.

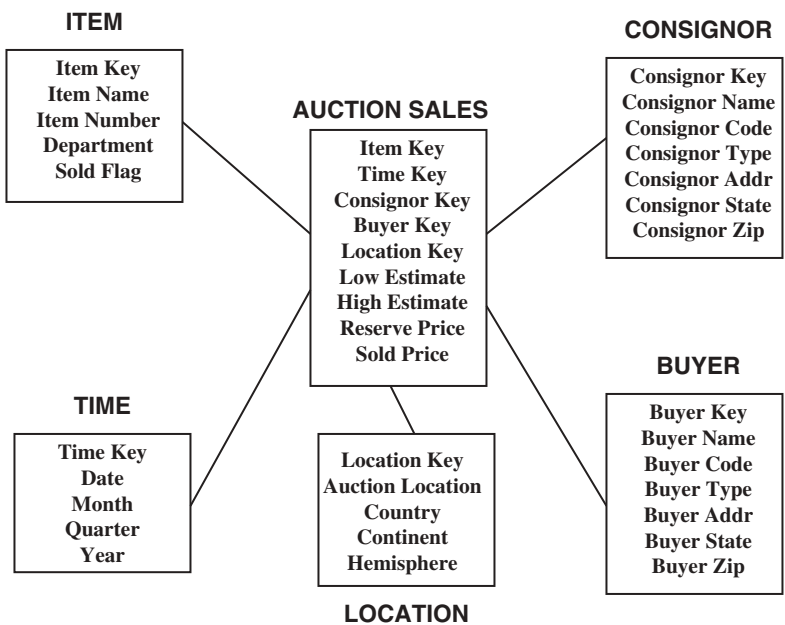


Figure 10-18 STAR schema example: auction company.

CHAPTER SUMMARY

- The components of the dimensional model are derived from the information packages in the requirements definition.

- The entity-relationship modeling technique is not suitable for data warehouses; the dimensional modeling technique is appropriate.
- The STAR schema used for data design is a relational model consisting of fact and dimension tables.
- The fact table contains the business metrics or measurements; the dimension tables contain the business dimensions. Hierarchies within each dimension table are used for drilling down to lower levels of data.
- STAR schema advantages are that it is easy for users to understand optimizes navigation, is most suitable for query processing, and enables specific performance schemes.

REVIEW QUESTIONS

1. Discuss the major design issues that need to be addressed before proceeding with the data design.
2. Why is the entity-relationship modeling technique not suitable for the data warehouse? How is dimensional modeling different?
3. What is the STAR schema? What are the component tables?
4. A dimension table is wide; the fact table is deep. Explain.
5. What are hierarchies and categories as applicable to a dimension table?
6. Differentiate between fully additive and semiadditive measures.
7. Explain the sparse nature of the data in the fact table.
8. Describe the composition of the primary keys for the dimension and fact tables.
9. Discuss data granularity in a data warehouse.
10. Name any three advantages of the STAR schema. Can you think of any disadvantages of the STAR schema?

EXERCISES

1. Match the columns:

- | | |
|-----------------------------|----------------------------------|
| 1. information package | A. enable drill down |
| 2. fact table | B. reference numbers |
| 3. case tools | C. level of detail |
| 4. dimension hierarchies | D. users understand easily |
| 5. dimension table | E. semiadditive |
| 6. degenerate dimensions | F. STAR schema components |
| 7. profit margin percentage | G. used for dimensional modeling |
| 8. data granularity | H. dimension attribute |
| 9. STAR schema | I. contains metrics |
| 10. customer demographics | J. wide |

2. Refer back to the information package given for a hotel chain in Chapter 5 (Fig. 5.6). Use this information package and design a STAR schema.
3. What is a factless fact table? Design a simple STAR schema with a factless fact table to track patients in a hospital by diagnostic procedures and time.
4. You are the data design specialist on the data warehouse project team for a manufacturing company. Design a STAR schema to track the production quantities. Production quantities are normally analyzed along the business dimensions of product, time, parts used, production facility, and production run. State your assumptions.
5. In a STAR schema to track the shipments for a distribution company, the following dimension tables are found: (1) time, (2) customer ship-to, (3) ship-from, (4) product, (5) type of deal, and (6) mode of shipment. Review these dimensions and list the possible attributes for each of the dimension tables. Also, designate a primary key for each table.